

Final Project

Decentralised Lending Service

REPORT

Peer2Peer Systems and Blockchains

Francesco Polperio 635406

Federico Lombardi xxxxxxx

5 June 2026

Contents

1	Assumptions	1
1.1	Specification interpretations	1
1.1.1	Compensation pool semantics	1
1.1.2	Tie at resolution	1
1.1.3	Rounding residue	1
1.1.4	Irreversible Failed status	1
1.1.5	Vote weight timing	1
1.2	Trust model	1
2	Project structure	2
2.1	On-chain contracts	2
2.1.1	LendingPool	2
2.1.2	LoanContract	2
2.1.3	BitcoinOracle	2
2.1.4	Auxiliary wrappers	2
2.2	Off-chain components	2
2.2.1	Oracle service	2
2.2.2	Automation scripts	3
2.3	Repository layout	3
2.4	Events	3
3	Implementation choices	4
3.1	Lending pool	4
3.1.1	State and accounting	4
3.1.2	Deposit and withdraw	4
3.1.3	Reentrancy protection	4
3.2	Proposal lifecycle	4
3.2.1	Submission and voting	4
3.2.2	Resolution	5
3.2.3	Proportional lock and ordering	5
3.3	Loan management	5
3.3.1	Disbursement	5
3.3.2	Partial repayment	5
3.3.3	Compensation	5
3.3.4	Termination	6
3.4	Upgradability	6
3.5	Oracle integration	6
4	Gas evaluation	7
4.1	Methodology	7
4.2	Results	7
4.3	Comments	7
4.3.1	Resolution dominates	7
4.3.2	Cheap reject paths	8
4.3.3	Deposits, withdraws, votes	8
4.3.4	Oracle update	8
4.3.5	Repayment	8

4.3.6	Compensation, termination, upgrade	8
5	Reentrancy modification and attack	9
5.1	Vulnerable variant	9
5.2	Attacker contract	9
5.3	Attack flow	9
5.4	Test	9
6	Malicious strategy	11
6.1	Whale self-loan with BTC drain	11
6.1.1	Spec ingredients	11
6.1.2	Numeric walkthrough	11
6.1.3	Final balance	12
6.1.4	Why honest defences fail	12
6.1.5	Variants	12
6.1.6	Mitigations	12
7	User manual	13
7.1	Requirements	13
7.2	Starting the private chain	13
7.3	Deployment and oracle	13
7.4	Demo	14
7.5	Automated voter	14
7.6	Tests and gas measurement	14

Chapter 1

Assumptions

1.1 Specification interpretations

1.1.1 Compensation pool semantics

The most significant choice concerns the semantics of the compensation pool: we interpret a compensation as a restoration of the contributor's disposable value on the funding pool (deposits left unchanged, `lockedValue` decreased by the paid amount), rather than as a direct ETH transfer to the contributor's wallet.

1.1.2 Tie at resolution

An exact tie between weighted accept and reject votes at resolution is treated as a rejection.

1.1.3 Rounding residue

Any rounding residue produced when computing the proportional contributor shares stays in the pool and is never credited to the applicant, so the actual loaned amount is always the sum of the locked shares.

1.1.4 Irreversible Failed status

The `Failed` status of a loan is irreversible: once a compensation request marks a loan as failed, any subsequent repayment can no longer flip it back to `Successful` and is routed entirely to the compensation pool.

1.1.5 Vote weight timing

Voting weights are evaluated at voting time on the current disposable value, not snapshotted at submission.

1.2 Trust model

The trust model assumes a well-behaved oracle operator, a well-behaved `LendingPool` owner for UUPS upgrades, and no proof of ownership or locking of the BTC funds declared as collateral by applicants. The last assumption is inherited from the project specification and is the root of the attack vector discussed in the malicious-strategy section.

Chapter 2

Project structure

2.1 On-chain contracts

The on-chain layer is split across three production contracts and two auxiliary wrappers.

2.1.1 LendingPool

LendingPool is the entry point of the service: it holds the global accounting of deposits and locked value, manages the proposal lifecycle (submission, voting, resolution), and deploys a new loan contract whenever a proposal is approved. It is deployed behind an ERC-1967 proxy and follows the UUPS upgradeability pattern, with `_authorizeUpgrade` gated by `onlyOwner` and a constructor-time `_disableInitializers` that prevents the implementation from being initialised directly.

2.1.2 LoanContract

LoanContract is instantiated once per approved loan: it freezes the per-contributor initial locks at construction, disburses the loaned amount to the applicant in the same transaction, and exposes `partialRepay`, `requestCompensation` and `terminate` as its public surface. Each instance is autonomous and references the deploying pool through an immutable address, so the pool-side mutator hooks can be gated by an `onlyLendingPool` modifier.

2.1.3 BitcoinOracle

BitcoinOracle stores the satoshi balance of a hashed Bitcoin address, accepts paid update requests, and is the only on-chain endpoint that can be written to by the off-chain operator. It exposes the conversion from satoshi balance to ether equivalent used by the pool during the BTC liquidity check at resolution time.

2.1.4 Auxiliary wrappers

LocalERC1967Proxy is a minimal wrapper around the OpenZeppelin proxy, needed because the private chain stops at the Berlin fork and does not support the `PUSH0` opcode introduced in Shanghai. **LendingPoolV2** is a mock target used only to exercise the UUPS upgrade path in the test suite: it appends a new storage slot and a new function on top of **LendingPool** and provides a `reinitializer(2)` migration hook.

2.2 Off-chain components

The off-chain layer comprises an oracle service and a set of Python automation scripts.

2.2.1 Oracle service

`oracle/oracle_service.py` parses the first 131,000 mainnet blocks from the local `blk*.dat` files, maintains a per-address satoshi UTXO map, listens for `UpdateRequested` events on the on-chain oracle con-

tract, and submits the corresponding **update** transactions signed by the operator key. The service is designed to process one block at a time and can be stopped and resumed without losing state.

2.2.2 Automation scripts

The scripts under **scripts/** cover the operational flow. **InitialSetup.py** creates EOAs, funds them from the prefunded miner account, deploys the production contracts behind the proxy and writes the resulting configuration files. **DemoOperations.py** runs a complete end-to-end scenario, printing the balances of the involved accounts and the relevant pool state after every meaningful step. **GasMeasurement.py** produces the gas report used in the dedicated section of this report. **YesMan.py** is the automated contributor required by the assignment: it watches **ProposalSubmitted** events and approves every new proposal as soon as it sees one.

2.3 Repository layout

The repository is organised so that each layer maps to a single top-level directory. **contracts/** contains the Solidity sources, split between the production code at the root, the deliberately vulnerable variant used for the reentrancy demonstration in **contracts/vulnerable/**, and the upgrade mock in **contracts/mocks/**. **scripts/** hosts all Python automation, **oracle/** hosts the off-chain oracle service, and **test/** contains the Hardhat test suites with one file per area (lending pool, loan lifecycle, proposals, resolution, oracle, oracle fee, reentrancy, upgradability, termination). **data/** collects all artefacts produced at runtime (account file, contract address files, gas report, oracle service state), **chaindata/** hosts the Bitcoin block files and the geth datadir, and **docs/** contains the assignment specification and this report.

2.4 Events

State transitions are exposed to off-chain consumers through a comprehensive set of events. The pool emits **Deposited** and **Withdrawn** on funding-pool movements, the full proposal lifecycle through **ProposalSubmitted**, **ProposalVoted**, **ProposalApproved** and **ProposalRejected**, and **LoanRegistered**, **LoanDeregistered**, **CollateralPercentageChanged** on the loan registry.

Each loan contract emits **LoanCreated** at construction, **Repayment** (with base, interest, excess and residuals) on every **partialRepay**, **MarkedFailed** on the transition to **Failed**, **CompensationRequested** (**owed**, **paid**) on each compensation attempt, **LoanClosed** on settlement, and **LoanTerminated** on **terminate**. The oracle emits **UpdateRequested** and **BalanceUpdated**. **ProposalSubmitted** is the event consumed by the **YesMan** automated voter, while **UpdateRequested** and **BalanceUpdated** are the integration points of the off-chain oracle service.

Chapter 3

Implementation choices

3.1 Lending pool

3.1.1 State and accounting

The pool tracks each contributor's total deposit in `deposits[]` and the portion currently locked in active loans in `lockedValue[]`; the difference, computed on demand by `disposableValue`, is the spendable balance used for withdrawals and vote weighting. The aggregates `totalFundingPool`, `totalLocked` and `compensationPool` mirror these per-contributor quantities at the global level, so that `totalDisposable` can be evaluated without iterating the contributor list. A separate `_contributorList` array, guarded by an anti-duplicate map, is appended on the first deposit of each address; it is iterated only at resolution time and entries with zero disposable value are skipped, which keeps the storage layout strictly append-only and upgrade-safe.

3.1.2 Deposit and withdraw

`deposit` accepts any value greater than or equal to $MIN_DEPOSIT = 100,000$ wei, registers the sender in the contributor list on first call, and updates both `deposits[]` and `totalFundingPool`. `withdraw` enforces a strict Checks-Effects-Interactions ordering: the disposable check is performed first, then `deposits[]` and `totalFundingPool` are decremented, and only at the end the ETH transfer is performed via a low-level `call`. The whole function sits behind the reentrancy mutex.

3.1.3 Reentrancy protection

External-call sites that move ETH out of the pool are protected by a manually managed single-slot mutex (`_reentrancyStatus`, initialised to 1 in `initialize` and toggled to 2 during a guarded call). This choice is preferred over OpenZeppelin's `ReentrancyGuardUpgradeable` to retain explicit control over the storage layout of the upgradeable contract; one slot is cheap. Paths that do not transfer ETH out of the pool, such as `compensateFromPool`, intentionally drop the guard because there is no reentrancy vector to defend against.

3.2 Proposal lifecycle

3.2.1 Submission and voting

`submitProposal` stores the applicant, the requested amount, the interest rate (constrained to $[1, 100]$), the duration, the keccak hash of the Bitcoin address, and the current block number. The proposal struct nests two mappings (`hasVoted` and `voteApprove`), so `_proposals` is kept `internal` and the public surface is exposed through explicit getters (`getProposal`, `hasVotedOn`, `getVoteApprove`) rather than relying on an auto-generated getter, which would refuse the nested mappings.

`vote` requires the caller to be a contributor, to have never voted on the proposal before, and the proposal to be still `Active`. Only approve voters are pushed into the iterable `approveVoters[]` array, while reject voters are tracked only through the `hasVoted` flag: this makes the resolution loop walk only the affirmative side.

3.2.2 Resolution

`resolveProposal` is callable only by the original applicant, only after `PROPOSAL_VOTING_PERIOD` = 12 blocks have elapsed since submission, and only while the proposal is still **Active**. The function checks, in order, that the pool has enough disposable value, that the Bitcoin liquidity reported by the oracle is at least equal to the requested amount, and that the weighted approval majority exceeds the strict threshold:

$$\text{weightedYes} \times 2 > \text{totalDisposable}$$

If any of these checks fails, the proposal transitions to **Rejected** and emits the corresponding event; otherwise it transitions to **Approved** and proceeds to the lock and deploy step.

3.2.3 Proportional lock and ordering

Each contributor's share is computed as:

$$\lfloor \text{amount} \times \text{disp}_i / \text{totalDisp} \rfloor$$

The actual loaned amount is the sum of these shares, so any rounding loss stays in the pool and is not credited to the applicant. Contributors are then sorted by share in descending order, with ties broken by ascending address, using an in-memory insertion sort: the algorithm runs entirely in memory (no `SSTORE`), the realistic number of contributors is small, and the resulting order is consumed verbatim by the per-loan contract to drive the highest-first refund rule. A private helper `_deployLoan` factors out the new `LoanContract{value: ...}` call to keep `resolveProposal` below Solidity's stack depth limit.

3.3 Loan management

3.3.1 Disbursement

`LoanContract` is deployed by the pool with `msg.value = loanedAmount`; its constructor records the contributor list, the per-contributor initial locks, the expected interest tranches (proportional to the initial locks), and immediately forwards the loaned amount to the applicant in the same transaction. This avoids a separate activation call and ensures that the disbursement and the lock state are committed atomically.

3.3.2 Partial repayment

`partialRepay` splits the incoming value between base and interest. The base component, capped at the remaining loan amount, is distributed by walking the pre-sorted contributors and refunding each one up to their residual capacity, defined as:

$$\text{initialLocked} - \text{unlockedSoFar} - \text{compRecovered}$$

If a contributor has already drawn from the compensation pool, the helper `_splitBaseForfeit` diverts a proportional slice of their share back to the compensation pool, implementing the spec rule that requesting a compensation forfeits the corresponding applicant repayment.

The interest component, capped at the remaining expected interest, is distributed proportionally to each contributor's initial lock; each tranche is then split between a gain part, sent directly to the contributor via `lendingPool.creditInterest`, and a collateral part equal to `collateralPercentage%` of the tranche, routed to the compensation pool. Any value exceeding loan plus interest is credited to the compensation pool. When the loan is fully repaid the contract sweeps any residual ETH to the compensation pool, marks itself **Successful** (only if it was not already **Failed**), calls `decreaseCollateral` on the pool and `markLoanClosed`.

3.3.3 Compensation

`requestCompensation` can be called by any contributor at any time while the loan is not **Successful**. On the first call after `expiryBlock` the status transitions to **Failed**, `increaseCollateral` fires on the pool, and the remaining expected interest is set to zero, so any later repayment goes entirely to the compensation pool. The compensation amount is computed as:

$$\min(\text{owed}, \text{compensationPool})$$

where the owed value subtracts both what the contributor has already been refunded by repayments and what has already been compensated. The actual movement, on the pool side, is a transfer from `lockedValue` back to `disposable` without any external call: the compensation pool decreases, the contributor's locked value decreases, the deposit is left unchanged. The call can therefore be invoked repeatedly as the compensation pool refills, and remains safe against an empty pool.

3.3.4 Termination

`terminate` is callable only when the loan is `Successful` or when every contributor on a `Failed` loan has been either fully repaid or fully compensated. It sweeps any residual ETH on the loan contract to the compensation pool, deregisters the loan from the pool through `markLoanClosed`, and sets an idempotent `terminated` flag so the call cannot run twice.

3.4 Upgradability

`LendingPool` is deployed behind an ERC-1967 proxy and follows the UUPS pattern. The `_authorizeUpgrade` hook is gated by `onlyOwner`, and the implementation constructor calls `_disableInitializers` so that the bare implementation contract cannot be initialised outside the proxy. The storage layout is strictly append-only, which is verified in the test suite against the `LendingPoolV2` mock: the mock appends one new slot and a new function on top of the v1 layout, and exposes a `reinitializer(2)` hook that ensures the v2 migration logic can run at most once after the v1 `initialize`.

3.5 Oracle integration

`BitcoinOracle` stores the satoshi balance of each registered address under the key `keccak256(btcAddress)`, exposes a paid `requestUpdate` that emits an event consumed by the off-chain service, and lets the operator push the resulting balance through `update`. The pool exposes `requestOracleUpdate` as a thin forwarder: it pulls the minimum fee from the oracle contract, requires the caller to attach at least that much value, and forwards the call.

Conversion from satoshi to wei is performed inside the oracle through the fixed BTC/ETH rate of 30, so the pool only needs to compare the returned ether equivalent with the requested loan amount at resolution time. The `MIN_ORACLE_FEE` constant is hardcoded at $28,603 \times 0.1$ gwei, with 28,603 being the measured gas cost of the warm-slot `update` on the private chain.

Chapter 4

Gas evaluation

4.1 Methodology

Gas costs are measured by `scripts/GasMeasurement.py` on the private geth chain, with gas price fixed at 1 gwei. Each scenario is run on a freshly initialised chain and the raw `gasUsed` value of each transaction receipt is collected. The full output is persisted in `data/gas_report.csv`.

4.2 Results

Operation (scenario)	Gas	Cost (ETH)
<code>deposit</code> – new contributor	138,734	1.39e-4
<code>deposit</code> – existing	40,064	4.01e-5
<code>withdraw</code> – partial	49,895	4.99e-5
<code>requestOracleUpdate</code> (via pool)	40,951	4.10e-5
<code>oracle.update</code> – cold slot	45,703	4.57e-5
<code>oracle.update</code> – warm slot	28,603	2.86e-5
<code>submitProposal</code>	184,376	1.84e-4
<code>vote</code> – approve	123,881	1.24e-4
<code>vote</code> – reject	59,653	5.97e-5
<code>resolveProposal</code> – Approved (N=2)	1,972,086	1.97e-3
<code>resolveProposal</code> – Approved (N=5)	2,346,916	2.35e-3
<code>resolveProposal</code> – Rejected (pool low)	63,139	6.31e-5
<code>resolveProposal</code> – Rejected (BTC)	73,084	7.31e-5
<code>resolveProposal</code> – Rejected (vote)	82,345	8.23e-5
<code>partialRepay</code> – mid	81,566	8.16e-5
<code>partialRepay</code> – close Successful	190,005	1.90e-4
<code>partialRepay</code> – overpay	192,831	1.93e-4
<code>partialRepay</code> – on Failed (split)	150,058	1.50e-4
<code>requestCompensation</code> – first (Failed)	85,212	8.52e-5
<code>requestCompensation</code> – subsequent	47,698	4.77e-5
<code>terminate</code> – Successful	27,696	2.77e-5
<code>upgradeToAndCall</code> – UUPS v2	37,692	3.77e-5

4.3 Comments

4.3.1 Resolution dominates

`resolveProposal` on the approve path is by far the most expensive operation in the system: it iterates the full contributor list, runs the insertion sort in memory, writes the per-contributor lock state on chain, and deploys a fresh `LoanContract` that immediately disburses the loaned amount. The bulk of the cost comes from the `CREATE` opcode and the constructor body, not from the sort. The delta between (N=2) and (N=5) is only about 375k gas, which confirms that the per-contributor overhead is small compared to the fixed deployment cost.

4.3.2 Cheap reject paths

The three rejection scenarios short-circuit before any deploy: insufficient pool, insufficient BTC liquidity and majority not reached. The vote-based rejection is the most expensive of the three because it still has to walk the affirmative voters to compute `weightedYes`; the pool-low and BTC checks return immediately after a single state read.

4.3.3 Deposits, withdraws, votes

`deposit` pays the cold-slot penalty on the first call (initialisation of `deposits[]` and registration in the contributor list); subsequent deposits drop to about a third of the cost. `withdraw` sits below 50k gas because all state slots are warm after the disposable check. Vote-approve is roughly twice as expensive as vote-reject because of the push into the iterable `approveVoters[]` array.

4.3.4 Oracle update

`oracle.update` paths follow standard gas constraints. The 17k gas gap between the cold and warm update paths matches the standard 20k `SSTORE` difference between writing a zero slot for the first time and overwriting a non-zero value. The pool-side `requestOracleUpdate` adds about 12k gas of forwarding logic on top of the bare oracle call.

4.3.5 Repayment

`partialRepay` cost grows with the number of contributors actually touched and with the closing branch: a mid-loan payment that refunds a single contributor sits around 80k gas, the close-Successful path crosses 190k because it triggers the final sweep, `decreaseCollateral` and `markLoanClosed`. The Failed-path variant is heavier than a comparable Active-path payment because `_splitBaseForfeit` runs on every iteration.

4.3.6 Compensation, termination, upgrade

The first `requestCompensation` after expiry is more expensive than subsequent ones because it transitions the loan to `Failed` and bumps `collateralPercentage`; later calls only update the per-contributor accounting. `terminate` on a clean Successful loan is the cheapest non-trivial operation, since the loan registry has already been cleared by the closing repayment. The UUPS upgrade path is similarly cheap: a single `SSTORE` on the implementation slot plus the `reinitializer(2)` call.

Chapter 5

Reentrancy modification and attack

5.1 Vulnerable variant

The vulnerable contract `contracts/vulnerable/LendingPoolVulnerable.sol` is a stripped clone of the production `LendingPool` with one deliberate change: in the `withdraw` function the external ETH transfer is performed *before* the state update, and the `nonReentrant` guard is removed. The Checks–Effects–Interactions ordering is therefore reversed, and the disposable balance check still passes when the call re-enters the function, because `deposits[]` has not yet been decremented.

```
function withdraw(uint256 amount) external {           // no nonReentrant
    require(amount > 0, "Zero_amount");
    require(disposableValue(msg.sender) >= amount, "Insufficient");
    (bool ok, ) = msg.sender.call{value: amount}(""); // INTERACTION first
    require(ok, "Transfer_failed");
    unchecked { deposits[msg.sender] -= amount;        // EFFECTS after
                totalFundingPool -= amount; }
}
```

5.2 Attacker contract

The attacker contract `contracts/vulnerable/ReentrancyAttacker.sol` is a minimal helper holding three pieces of state: the address of the vulnerable pool, the owner, the recorded `attackAmount` and an attack counter. It exposes a `deposit` function that forwards the value to the vulnerable pool to satisfy the disposable check, an `attack` function callable only by the owner that triggers the first `withdraw`, a `receive` fallback that re-enters `withdraw` as long as the counter has not reached `MAX_REENTRIES = 5` and the pool still holds enough balance, and a `sweep` function used by the owner to recover the drained ETH after the attack.

5.3 Attack flow

The attacker first calls `deposit` with the chosen stake, which records `attackAmount` and registers the stake on the vulnerable pool. The owner then calls `attack`, which invokes `withdraw(attackAmount)` on the pool. The pool transfers the ETH back to the attacker through a low-level `call`; the attacker's `receive` function then re-enters `withdraw` with the same amount. Because the pool has not yet decremented its bookkeeping, the disposable check still succeeds, and the loop continues until either the maximum number of re-entries is reached or the pool runs out of balance. At the end of the cascade, the attacker has withdrawn the original stake plus the drained value, while the funds of other contributors become unreachable on the pool.

5.4 Test

The test `test/Reentrancy.test.js` wires the vulnerable pool against the real oracle, seeds the pool with legitimate deposits from a few honest contributors, deploys the attacker, runs the attack, and

asserts that the attacker's net ETH gain exceeds its original stake. The same test then targets the production `LendingPool` with an identical attacker and verifies that the attack reverts with "**Reentrant call**", confirming that the mutex and the Checks-Effects-Interactions ordering on the production code prevent the exploit.

Chapter 6

Malicious strategy

6.1 Whale self-loan with BTC drain

The strategy combines four features of the specification into a profitable extraction of value from honest contributors. None of the steps relies on reentrancy; all of them are explicitly permitted by the specification.

6.1.1 Spec ingredients

The first ingredient is the absence of any per-voter weight cap: the resolution rule approves a proposal as soon as the weighted yes votes exceed half of the disposable pool, so a single contributor holding more than half of the disposable value can unilaterally approve any proposal, including their own.

The second ingredient is that the specification explicitly allows the same address to act as applicant and contributor on the same loan.

The third ingredient is the highest-locked-first rule on partial repayments: contributors are refunded in descending order of initial locked value, with ties broken by ascending address, so whoever locked the largest share is reimbursed first.

The fourth ingredient is the absence of any ownership proof or locking on the BTC collateral: the applicant can pledge any rich Bitcoin address, the oracle balance is a one-shot snapshot, and there is no clawback if the BTC is moved after the resolution.

6.1.2 Numeric walkthrough

Consider an initial pool with Alice holding 6 ETH (60%), and three honest contributors c_1 , c_2 , c_3 holding respectively 1, 1 and 2 ETH, for a total disposable value of 10 ETH. Alice also controls a Bitcoin address holding 1 BTC, which corresponds to 30 ETH at the spec rate.

She starts by requesting an oracle update for the hash of her BTC address paying the minimum fee, so that the on-chain oracle records 1 BTC for that hash. She then submits a loan proposal for 10 ETH, with rate 10, duration 100 and the same BTC hash. She votes approve, which contributes her full 6 ETH of disposable value to the weighted yes count.

After at least thirteen blocks she calls `resolveProposal`: the pool sufficiency check passes ($10 \geq 10$), the BTC liquidity check passes ($30 \geq 10$), the weighted majority check passes ($6 \times 2 = 12 > 10$). The proposal moves to Approved and the proportional lock is computed: 6 from Alice, 2 from c_3 , 1 from c_1 and 1 from c_2 . The insertion sort produces the order Alice, c_3 , c_1 , c_2 . A fresh `LoanContract` is deployed with `msg.value = 10`, and its constructor disburses 10 ETH directly to Alice's wallet.

Once the disbursement is on her wallet, Alice moves the 1 BTC away from the pledged address. The Ethereum side is unaffected: the loan has already been granted and the protocol has no clawback bound to the oracle balance. Alice then calls `partialRepay` with 6 ETH. The base loop iterates the pre-sorted contributors and immediately picks Alice as the first refund target: her residual capacity is 6, the helper `_splitBaseForfeit` returns (0, 6), her `unlockedSoFar` is incremented to 6 and `lendingPool.repayLockedValue` is called with 6, which brings her `lockedValue` to zero. The base remaining drops to zero and the loop exits before any of the honest contributors is reached.

Alice deliberately does not pay any interest: under the initial collateral percentage of 50, half of every interest tranche would be redistributed across the contributors proportionally to their initial locks, which

would partially undo her gain. The loan remains Active with a residual loan amount of 4 ETH. After the duration elapses, the honest contributors call `requestCompensation`: the loan transitions to Failed, the collateral percentage is bumped up, but the compensation pool is empty because no successful loan has ever fed it, so each honest call recovers zero. Finally Alice withdraws her now-unlocked 6 ETH deposit.

6.1.3 Final balance

Alice’s net cashflow on the ETH side is -6 on the deposit, $+10$ on the disbursement, -6 on the self-repayment and $+6$ on the withdrawal, for a net gain of $+4$ ETH. Her BTC holding is intact. The honest contributors lose 1, 1 and 2 ETH respectively, for a total loss of 4 ETH. The transfer is exactly zero-sum and the operational cost is negligible: the gas consumed across the seven involved transactions amounts to approximately 0.005 ETH on the test chain, the oracle fee is well below 10^{-5} ETH, the elapsed wall-clock time is about twelve Ethereum blocks plus one Bitcoin transaction.

6.1.4 Why honest defences fail

Voting no does not change anything because Alice already controls a weighted majority on her own. A pre-resolve withdrawal by honest contributors would lower the total disposable value below the requested amount and trigger the pool-insufficiency early reject, but Alice can defeat this by depositing fresh ETH right before her own `resolveProposal` call, since voting weights are evaluated at resolution time on the current disposable value and not snapshotted at submission.

Refreshing the oracle before the resolution is useless because Alice moves the BTC only after the disbursement; refreshing after the resolution is useless because the loan contract has no oracle-driven clawback. Honest contributors cannot independently verify which Bitcoin address she pledged, because the proposal stores only the keccak hash of the address and the preimage cannot be recovered in polynomial time. There is no on-chain blacklist, so even if she were identified ex-post she could repeat the attack from a fresh wallet.

6.1.5 Variants

In the no-BTC-ownership variant the attacker never holds the BTC at all: she picks any public address with sufficient balance, for instance an exchange hot wallet, and uses its hash as the proof of liquidity; the specification explicitly allows this, and the profit is unchanged.

In the skip-self-repay variant the attacker simply lets the loan default without ever paying back: her deposit of 6 ETH remains locked forever inside the pool, but the disbursement still nets her $+4$ ETH; the same profit is achieved with less effort but with ETH frozen on chain.

In the collusion variant two distinct wallets jointly hold more than half of the disposable pool and split the load across parallel proposals to make detection harder.

In the Sybil variant a single attacker splits the 6 ETH across many small wallets to evade naive whale detection, with no change in the underlying economy.

6.1.6 Mitigations

A per-voter weight cap, for instance limiting the weight of a single address to thirty per cent of the total disposable value, blocks the unilateral approval directly. Banning the case applicant equals contributor raises the bar without a multi-wallet split. Replacing the highest-locked-first refund with a strictly pro-rata distribution eliminates the front-of-queue advantage that lets the attacker recover her own deposit before honest contributors. Treating the applicant’s own contributor share as junior in the failed case, or slashing it explicitly on failure, removes the incentive to act as both roles on the same loan.

A BTC-side HTLC, even if it would require a federation or a separate sidechain, would close the collateral hole. Snapshotting the voting weights at submission time, rather than at resolution, defeats the sandwich-deposit defence bypass. Forcing a fresh oracle refresh within a short window before the resolution narrows the staleness gap. Initialising the collateral percentage to a higher value would also make any partial-repayment strategy less profitable for an attacker who insists on paying interest. The minimal-invasive combination of a voting cap and a pro-rata refund rule already removes both economic preconditions of the attack without requiring any cross-chain machinery.

Chapter 7

User manual

7.1 Requirements

The code targets a Linux environment with `node` (version 18 or higher, with `npm install` already run at the project root), Python 3.10 or higher inside a virtual environment with the dependencies of `requirements.txt` installed, the `geth` binary at Berlin fork compatibility, and `jq` available on the path. The first `blk00000.dat` and `blk00001.dat` files of the Bitcoin mainnet must be placed inside the `chaindata/` directory, since the off-chain oracle reads them to compute address balances over the first 131,000 blocks.

7.2 Starting the private chain

The first terminal is dedicated to the `geth` node. The data directory must be wiped, then re-initialised from the genesis file shipped with the project, and finally the node must be started with the miner unlocked on the prefunded account. The full sequence is:

```
geth --datadir data removedb
geth --datadir data init project2526genesis.json
geth --datadir data --networkid 202526 \
  --http --http.api eth,net,web3,personal,debug,admin \
  --http.corsdomain '*' --allow-insecure-unlock \
  --nodiscover --maxpeers 0 \
  --mine --miner.gaslimit 30000000 \
  --mine.etherbase 0xd278d247A52C550508ea2b2C9321d816238fb523 \
  --unlock 0xd278d247A52C550508ea2b2C9321d816238fb523 \
  --password 0xd278d247A52C550508ea2b2C9321d816238fb523psw.txt
```

The node must be left running for the rest of the session.

7.3 Deployment and oracle

The second terminal performs the initial deployment and runs the off-chain oracle. After activating the Python virtual environment, the contracts are compiled and the deployment script is run; once the deployment has written the configuration files under `data/`, the oracle service can be started in the same terminal:

```
source venv/bin/activate
npx hardhat compile
python3 scripts/InitialSetup.py
OPERATOR_PRIVATE_KEY=$(jq -r .oracle_operator.key data/accounts.json) \
python3 oracle/oracle_service.py
```

The oracle service must stay running for the duration of any session that involves loan proposals, since the resolution step reads the balance written on chain by the operator.

7.4 Demo

The third terminal runs the demonstration script, which exercises a complete end-to-end scenario and prints the balances and the relevant pool state after every meaningful operation. The virtual environment must be activated again in this terminal, then the script is launched:

```
source venv/bin/activate
python3 scripts/DemoOperations.py
```

7.5 Automated voter

If the run is meant to include the automated voter required by the assignment, the voter must be started *before* the demo: open a dedicated terminal, activate the virtual environment, and launch **YesMan.py**; once the voter is listening for **ProposalSubmitted** events, start the demo in its own terminal. The voter automatically deposits a small stake on the pool on startup and approves every new proposal it observes.

```
source venv/bin/activate
python3 scripts/YesMan.py
```

7.6 Tests and gas measurement

The full Hardhat test suite is launched from the project root with `npx hardhat test`; individual files can be targeted by appending the path, for instance `npx hardhat test test/Reentrancy.test.js` to exercise the reentrancy demonstration against the deliberately vulnerable contract and against the production pool. The gas table reported in this document is regenerated by `python3 scripts/GasMeasurement.py` against the private chain; the script produces `data/gas_report.csv`. The exact geth re-initialisation sequence used to obtain repeatable measurements is documented in `docs/Instruction.txt`.